

# **Global Engineering Solutions**

## **FEST<sup>®</sup>**

**FEST Library Reference Manual**

|    |                      |         |
|----|----------------------|---------|
| 1. | OnInitDevice .....   | page 2  |
| 2. | OnKillDevice .....   | page 4  |
| 3. | OnReadDigital .....  | page 5  |
| 4. | OnWriteDigital ..... | page 6  |
| 5. | OnReadAnalog .....   | page 7  |
| 6. | OnWriteAnalog .....  | page 8  |
| 7. | OnReadString .....   | page 9  |
| 8. | OnWriteString .....  | page 10 |



# 1. OnInitDevice

## ● Description

- Driver가 Load 될 때 해당하는 DLL File을 찾아 Memory에 Loading한 후 OnInitDevice 를 수행된다.
- DLL 이 Memory 에 Loading 될 때 오직 한번만 수행하는 API 이며, 주로 Driver Initialize 에 사용된다.

## ● Syntax

- `BOOL OnInitDevice ( int id1, int id2, int id3, int id4, int id5, int id6, int id7, int id8, int id9, int id10, TCHAR *szParm , void **ppDrvData /*OUT*/ );`

## ● Parameter

- ppDrvData = Digital/Analog/String IO에서 Interface를 위해 사용할 Driver 포인터
- szParm = Driver DLL Loading 시 Parameter로 넘겨받아 사용할수 있는 user parameter(예, driver name)
- id1 = Driver DLL Loading 시 초기화를 위한 1 Parameter(ComPort no)
- id2 = Driver DLL Loading 시 초기화를 위한 2 Parameter(Baudrate)
- id3 = Driver DLL Loading 시 초기화를 위한 3 Parameter(Data bit)
- id4 = Driver DLL Loading 시 초기화를 위한 4 Parameter(Stop bit)
- id5 = Driver DLL Loading 시 초기화를 위한 5 Parameter(Parity)
- id6 = Driver DLL Loading 시 초기화를 위한 6 Parameter
- id7 = Driver DLL Loading 시 초기화를 위한 7 Parameter
- id8 = Driver DLL Loading 시 초기화를 위한 8 Parameter
- id9 = Driver DLL Loading 시 초기화를 위한 9 Parameter
- id10 = Driver DLL Loading 시 초기화를 위한 10 Parameter(Communication Timeout)

## ● Return

- 성공적으로 수행 했을 때는 TRUE, 그렇지 않을 때는 FALSE 를 return 한다.
- FALSE 로 return 되었을 때는 Driver Initialize 실패로 더 이상 Main Program 이 진행되지 않고 종료된다.

## ● Sample

```

BOOL OnInitDevice( int id1 , int id2 , int id3 , int id4 , int id5 , int id6 , int id7 , int id8 , int id9 , int id10,
TCHAR *szParm , void **ppDrvData /*OUT*/ ) {
    TComSerial *pcComSerial = new TComSerial( id1 , id2 , id3 , id4 , id5 );
    *ppDrvData = pcComSerial;

    if ( ID10 < 100 ) TimeOut = 100;           //communication timeout
    else              TimeOut = ID10;

```

```
if ( !pcComSerial->Initialize() ) {  
    printf("====> [ROBOT-DRIVER] RS-232 PORT[%d] Initialize() ERROR...return false\n", ID1);  
    return FALSE;  
}  
if ( !pcComSerial->Connect() ) {  
    printf("====> [ROBOT-DRIVER] RS-232 PORT[%d] Connect() ERROR...return false\n", ID1);  
    return FALSE;  
}  
return TRUE;  
}
```

---

## 2. OnKillDevice

---

### ● Description

- S/W 종료시 driver unload로 될 때 OnKillDevice 를 수행하고 Memory로부터 unloading하게 된다.
- DLL이 Memory로 부터 Unloading 하기 전 오직 한번만 수행하는 API이며, 주로 Driver DeInitialize에 사용된다.

### ● Syntax

```
BOOL OnKillDevice ( void* pDrvData , int id1, int id2, int id3, int id4, int id5, int id6, int id7, int id8, int id9,  
int id10, TCHAR *szParm );
```

### ● Parameter

- OnInitDevice() Part 와 동일(Same with Initialize routine.)

### ● Return

- 성공적으로 수행 했을 때는 TRUE, 그렇지 않을 때는 FALSE 를 return 한다.

### ● Sample

```
BOOL OnKillDevice( void* pDrvData , int id1 , int id2 , int id3 , int id4 , int id5 , int id6 , int id7 , int id8 , int  
id9 , int id10, TCHAR *szParm ) {  
    delete ((TComSerial *)pDrvData);  
    return TRUE;  
}
```

## 3. OnReadDigital

### ● Description

- 정의한 Digital IO 의 Input(DI)쪽에 관련된 API 로써 Digital IO 의 Input Data 를 읽으려고 할 때 이 API 가 불러지게 된다

### ● Syntax

- int OnReadDigital ( void\* pDrvData , void\* , int id1, int id2, int id3, int id4, int\* Result );

### ● Parameter

- Driver IO를 정의한 cfg 파일의 DI(Digital Input) id를 넘겨 받는다
- pDrvData = IO Driver 포인터
- id1 = IO Handling DRIVER 와 통신을 위한 1 Parameter('-' 로 정의시 -1 )
- id2 = IO Handling DRIVER 와 통신을 위한 2 Parameter('-' 로 정의시 -1 )
- id3 = IO Handling DRIVER 와 통신을 위한 3 Parameter('-' 로 정의시 -1 )
- id4 = IO Handling DRIVER 와 통신을 위한 4 Parameter('-' 로 정의시 -1 )
- Result = IO Reading(통신)의 성공여부를 나타내는 Parameter 로, 성공적으로 Reading 했을 때는 TRUE, 그렇지 않을 때는 FALSE 를 Setting 하면 된다

### ● Return

- 요구에 따른 Digital Input의 값을 return 한다.
- Parameter "int \*Result" 가 FALSE일 때는 return 값은 의미가 없다.  
(Memory에 Update 되지 않고 이전 상태를 유지하게 된다)

### ● Sample

```
int OnReadDigital( void* pDrvData , void* , int id1, int id2, int id3, int id4, int* Result ){
    int Data=0;
    *Result = FALSE;
    ...
    Data=Controller 와 통신하여 얻은 값;
    ...
    *Result = Controller 통신 상태;
    return Data;
}
```

## 4. OnWriteDigital

### ● Description

- Digital IO 의 Output(DO) 쪽에 관련된 API 로써 Digital IO 의 Output 에 Data 를 쓰려고 할 때 이 API 가 불려지게 된다.

### ● Syntax

- void OnWriteDigital ( void\* pDrvData , void\* , int id1, int id2, int id3, int id4, int SetValue , int\* Result );

### ● Parameter

- Driver IO를 정의한 cfg 파일의 DO(Digital Output) id를 넘겨 받는다
- pDrvData = IO Driver 포인터
- id1 = IO Handling DRIVER 와 통신을 위한 1 Parameter('-' 로 정의시 -1 )
- id2 = IO Handling DRIVER 와 통신을 위한 2 Parameter('-' 로 정의시 -1 )
- id3 = IO Handling DRIVER 와 통신을 위한 3 Parameter('-' 로 정의시 -1 )
- id4 = IO Handling DRIVER 와 통신을 위한 4 Parameter('-' 로 정의시 -1 )
- SetValue = 쓰려고 하는 Data (Integer)
- Result = IO Writing(통신)의 성공여부를 나타내는 Parameter 로, 성공적으로 Writing 했을 때는 TRUE, 그렇지 않을 때는 FALSE 를 Setting 하면 된다.

### ● Return

Nothing

### ● Sample

```
void OnWriteDigital( void* pDrvData , void* , int id1, int id2, int id3, int id4, int SetValue , int* Result ) {
    *Result = FALSE;
    ...
    SetValue 를 Controller 에 Write
    ...
    *Result = Controller 통신 상태;
}
```

## 5. OnReadAnalog

### ● Description

- 정의한 Analog IO 의 Input (AI)쪽에 관련된 API 로써 Analog IO 의 Input Data 를 읽으려고 할 때 이 API 가 불러지게 된다.

### ● Syntax

- double OnReadAnalog ( void\* pDrvData , void\* , int id1, int id2, int id3, int id4, int\* Result );

### ● Parameter

- Driver IO를 정의한 cfg 파일의 AI(Analog Input) id를 넘겨 받는다
- pDrvData = IO Driver 포인터
- id1 = IO Handling DRIVER 와 통신을 위한 1 Parameter('-' 로 정의시 -1 )
- id2 = IO Handling DRIVER 와 통신을 위한 2 Parameter('-' 로 정의시 -1 )
- id3 = IO Handling DRIVER 와 통신을 위한 3 Parameter('-' 로 정의시 -1 )
- id4 = IO Handling DRIVER 와 통신을 위한 4 Parameter('-' 로 정의시 -1 )
- Result = IO Reading(통신)의 성공여부를 나타내는 Parameter 로, 성공적으로 Reading 했을 때는 TRUE, 그렇지 않을 때는 FALSE 를 Setting 하면 된다.

### ● Return

- 요구에 따른 Analog Input 의 값을 return 한다.
- Parameter "int \*Result" 가 FALSE일 때는 return 값은 의미가 없다.  
(Memory에 Update 되지 않고 이전 상태를 유지하게 된다)

### ● Sample

```
double OnReadAnalog( void* pDrvData , void* , int id1, int id2, int id3, int id4, int* Result ) {
    double dData=0;
    *Result = FALSE;

    ...
    dData=Controller 와 통신하여 얻은 값;
    ...
    *Result = Controller 통신 상태;
    return dData;
}
```



## 6. OnWriteAnalog

### ● Description

- Analog IO 의 Output (AO)쪽에 관련된 API 로써 Analog IO 의 Output 에 Data 를 쓰려고 할 때 이 API 가 불러지게 된다.

### ● Syntax

- void OnWriteAnalog( void\* pDrvData , void\* , int id1, int id2, int id3, int id4, double SetValue , int\* Result );

### ● Parameter

- Driver IO를 정의한 cfg 파일의 AO(Analog Output) id를 넘겨 받는다
- pDrvData = IO Driver 포인터
- id1 = IO Handling DRIVER 와 통신을 위한 1 Parameter('-' 로 정의시 -1 )
- id2 = IO Handling DRIVER 와 통신을 위한 2 Parameter('-' 로 정의시 -1 )
- id3 = IO Handling DRIVER 와 통신을 위한 3 Parameter('-' 로 정의시 -1 )
- id4 = IO Handling DRIVER 와 통신을 위한 4 Parameter('-' 로 정의시 -1 )
- Data = 쓰려고 하는 Data (double)
- Result = IO Writing(통신)의 성공여부를 나타내는 Parameter 로, 성공적으로 Writing 했을 때는 TRUE, 그렇지 않을 때는 FALSE 를 Setting 하면 된다.

### ● Return

Nothing

### ● Sample

```
void OnWriteAnalog( void* pDrvData , void* , int id1, int id2, int id3, int id4, double SetValue , int* Result ) {
    *Result = FALSE;
    ...
    SetValue 를 Controller 에 Write
    ...
    *Result = Controller 통신 상태;
}
```

## 7. OnReadString

### ● Description

- String IO 의 Input (SI)쪽에 관련된 API 로써 String IO 의 Input Data 를 읽으려고 할 때 이 API 가 불려지게 된다.

### ● Syntax

- void OnReadString( void\* pDrvData , void\* , int id1, int id2, int id3, int id4, TCHAR\* pszRtn , int\* Result );

### ● Parameter

- Driver IO를 정의한 cfg 파일의 SI(String Input) id를 넘겨 받는다
- pDrvData = IO Driver 포인터
- id1 = IO Handling DRIVER 와 통신을 위한 1 Parameter('-' 로 정의시 -1 )
- id2 = IO Handling DRIVER 와 통신을 위한 2 Parameter('-' 로 정의시 -1 )
- id3 = IO Handling DRIVER 와 통신을 위한 3 Parameter('-' 로 정의시 -1 )
- id4 = IO Handling DRIVER 와 통신을 위한 4 Parameter('-' 로 정의시 -1 )
- pszRtn = Reading 한 값
- Result = IO Reading(통신)의 성공여부를 나타내는 Parameter 로, 성공적으로 Reading 했을 때는 TRUE, 그렇지 않을 때는 FALSE 를 Setting 하면 된다.

### ● Return

Nothing

### ● Sample

```
void OnReadString( void* pDrvData , void* , int id1, int id2, int id3, int id4, TCHAR* pszRtn , int* Result ) {
    *Result = FALSE;
    ...
    pszRtn =Controller 와 통신하여 얻은 값;
    ...
    *Result = Controller 통신 상태;
}
```

## 8. OnWriteString

### ● Description

- String IO 의 Output (SO)쪽에 관련된 API 로써 String IO 의 Output 에 Data 를 쓰려고 할 때 이 API 가 불러지게 된다.

### ● Syntax

- void OnWriteString ( void\* pDrvData , void\* , int id1, int id2, int id3, int id4, TCHAR\* SetValue , int\* Result );

### ● Parameter

- Driver IO를 정의한 cfg 파일의 SO(String Output) id를 넘겨 받는다
- pDrvData = IO Driver 포인터
- id1 = IO Handling DRIVER 와 통신을 위한 1 Parameter('-' 로 정의시 -1 )
- id2 = IO Handling DRIVER 와 통신을 위한 2 Parameter('-' 로 정의시 -1 )
- id3 = IO Handling DRIVER 와 통신을 위한 3 Parameter('-' 로 정의시 -1 )
- id4 = IO Handling DRIVER 와 통신을 위한 4 Parameter('-' 로 정의시 -1 )
- SetValue = 쓰려고 하는 Data (String)
- Result = IO Writing(통신)의 성공여부를 나타내는 Parameter 로, 성공적으로 Writing 했을 때는 TRUE, 그렇지 않을 때는 FALSE 를 Setting 하면 된다.

### ● Return

Nothing

### ● Sample

```
void OnWriteString( void* pDrvData , void* , int id1, int id2, int id3, int id4, TCHAR* SetValue , int* Result ) {
    *Result=FALSE;
    ...
    SetValue 를 Controller 에 Write
    ...
    *Result = Controller 통신 상태;
}
```

---

URL : <http://www.gesolutions.co.kr/>

(주)게스

우 305-500

대전광역시 유성구 용산동 533-1번지 미건테크노월드Ⅱ C동 440호

전화 : 042-671-3331

G.E.S

#440C-Gate Migun TechnoWorld Ⅱ, 533-1

Yongsan-dong, Yuseong-gu, Daejeon, Korea

Tel : +82-42-671-3331

